

Cloud-Based Security Operations Center using Wazuh SIEM

Project Implementation Report

Done By:

T.R.J.Swarup Reddy

DECLARATION

I hereby declare that this report titled "Cloud-Based Security Operations Center (SOC) Using Wazuh SIEM on Microsoft Azure" is my own work. The implementation, configuration, testing, and documentation presented in this report were carried out by me as part of my cybersecurity learning and portfolio development.

The project has been developed using publicly available documentation and open-source technologies. All external resources used during the implementation have been appropriately acknowledged in the References section.

Name: T. R. J. Swarup Reddy

Date: 29th July 2026

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to the developers and contributors of the Wazuh Project, Microsoft Azure, Microsoft Sysinternals, and VirusTotal for providing the open-source technologies, cloud services, and technical documentation that made this project possible.

I also acknowledge the official documentation, technical guides, and community resources that assisted me throughout the deployment, configuration, troubleshooting, and validation phases of this implementation. These resources played a significant role in developing a practical understanding of Security Information and Event Management (SIEM) technologies and cloud-based security monitoring.

Finally, I would like to thank my mentors, instructors, and peers whose encouragement and guidance motivated me to strengthen my practical cybersecurity skills through hands-on implementation of this project.

INDEX	05
1. Introduction	06
2. Project Objectives	
3. Project Architecture	
4. Technology Stack	
5. Azure Infrastructure Setup	08
5.1 Objective	
5.2 Infrastructure Overview	
5.3 Azure Configuration	
5.4 Implementation	
5.5 Validation	
5.6 Outcome	
6. Wazuh Manager Installation	11
6.1 Objective	
6.2 Installation Overview	
6.3 Component Description	
6.4 Configuration	
6.5 Validation	
6.6 Outcome	
7. Ubuntu Agent Deployment	13
7.1 Objective	
7.2 Endpoint Overview	
7.3 Agent Installation	
7.4 Configuration	
7.5 Validation	
7.6 Outcome	
8. File Integrity Monitoring (FIM)	15
8.1 Objective	
8.2 Overview	

8.3 Configuration	
8.4 Validation	
8.5 Results	
8.6 Outcome	
9. VirusTotal Integration	18
9.1 Objective	
9.2 Overview	
9.3 Configuration	
9.4 Validation	
9.5 Results	
9.6 Outcome	
10. Windows Endpoint Monitoring	21
10.1 Objective	
10.2 Windows Agent Deployment	
10.3 Sysmon Integration	
10.4 Windows File Integrity Monitoring	
10.5 Validation	
10.6 Outcome	
11. Challenges Encountered	28
12. Results and Discussion	29
13. Conclusion and Future Enhancements	30
13.1 Conclusion	
13.2 Future Enhancements	
14. References	31

1. Introduction

Modern organizations generate a large volume of security events across their infrastructure, making it difficult to manually identify malicious activity. A Security Information and Event Management (SIEM) solution helps centralize log collection, monitor system activity, and detect potential threats in real time.

This project demonstrates the implementation of a cloud-hosted Security Operations Center (SOC) using **Wazuh** deployed on **Microsoft Azure**. The environment consists of a Wazuh Manager hosted on an Ubuntu Server virtual machine and monitored endpoints connected through Wazuh agents.

To improve threat detection capabilities, **File Integrity Monitoring (FIM)** is configured to monitor changes within a designated directory. Whenever a monitored file is created or modified, Wazuh calculates its hash and queries the **VirusTotal** threat intelligence platform. If the file is identified as malicious, Wazuh generates an enriched security alert containing the VirusTotal detection results.

The objective of this project is to build a practical SOC environment capable of monitoring endpoints, detecting suspicious file activity, and validating malware detection using real-world security workflows.

2. Project Objectives

The primary objective of this project is to design and implement a practical Security Operations Center (SOC) environment using Wazuh for endpoint monitoring and malware detection.

The project focuses on achieving the following objectives:

- Deploy a Wazuh Manager on a Microsoft Azure Ubuntu Server.
- Connect Linux and Windows endpoints using Wazuh agents.
- Configure File Integrity Monitoring (FIM) to detect file creation, modification, deletion, and renaming events.
- Integrate Wazuh with VirusTotal to enrich file events with threat intelligence.
- Validate malware detection using the EICAR test file.
- Build a centralized dashboard for monitoring security events and alerts.
- Gain practical experience with SIEM deployment, endpoint monitoring, and threat detection workflows.

3. Project Architecture

The project follows a simple centralized architecture where all monitored endpoints send security events to a single Wazuh Manager hosted on Microsoft Azure.

Component	Purpose
Azure Ubuntu Server	Hosted the Wazuh Manager, Indexer, and Dashboard
Wazuh Manager	Collected, processed, and correlated security events
Wazuh Dashboard	Provided a centralized interface for monitoring alerts and endpoint activity
Ubuntu Agent	Monitored Linux endpoint activity and forwarded security events
Windows Agent	Monitored Windows endpoint activity and forwarded security events
VirusTotal	Enriched file integrity events with threat intelligence by analysing file hashes



Architecture Overview

The Wazuh Manager is deployed on an Ubuntu Server virtual machine in Microsoft Azure. Endpoints communicate securely with the manager through Wazuh agents.

When a file is created or modified within a monitored directory, the agent sends the event to the Wazuh Manager. The manager computes the file hash and queries VirusTotal. If the file is recognised as malicious, the VirusTotal results are added to the alert and displayed in the Wazuh Dashboard for analysis.

4. Technology Stack

The project was implemented using a combination of cloud services, operating systems, security monitoring tools, and threat intelligence platforms. Each component was selected to provide a complete Security Operations Center (SOC) environment capable of monitoring endpoints, detecting malicious files, and supporting incident response.

Component	Technology Used	Purpose
Cloud Platform	Microsoft Azure	Hosted the Wazuh server infrastructure
Server Operating System	Ubuntu Server 22.04 LTS	Hosted the Wazuh Manager, Indexer, and Dashboard
Client Operating Systems	Ubuntu Desktop, Windows 10/11	Simulated monitored endpoints
SIEM Platform	Wazuh	Centralized log collection, event analysis, and security monitoring
Endpoint Monitoring	Wazuh Agent	Collected logs and monitored endpoint activities
Windows Monitoring	Sysmon	Generated detailed Windows system activity logs
Threat Intelligence	VirusTotal API	Checked file hashes against known malware databases

5. Azure Infrastructure Setup

5.1 Objective

The objective of this phase was to deploy a secure cloud environment capable of hosting the Wazuh SIEM infrastructure. Microsoft Azure was selected to provide a reliable and scalable platform for hosting the Wazuh Manager, Indexer, and Dashboard.

5.2 Infrastructure Overview

A dedicated Ubuntu Server virtual machine was deployed in Microsoft Azure. This virtual machine hosted all core Wazuh components required for centralized security monitoring.

The deployment included:

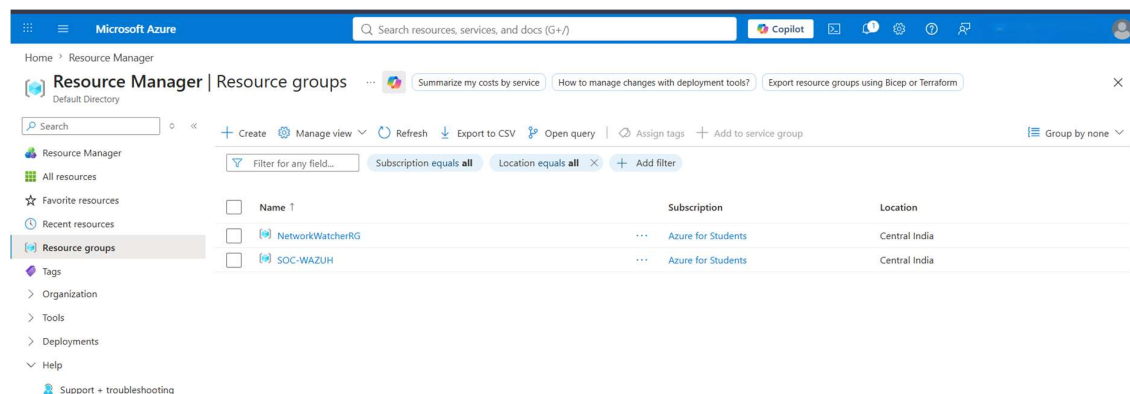
- Ubuntu Server 22.04 LTS
- Wazuh Manager
- Wazuh Indexer
- Wazuh Dashboard

Hosting these services on a cloud-based virtual machine allowed monitored endpoints to securely communicate with the Wazuh server over the internet while providing centralized access to the monitoring dashboard.

5.3 Azure Configuration

The following Azure resources were created for the deployment.

Resource	Configuration
Cloud Platform	Microsoft Azure
Virtual Machine	Ubuntu Server 22.04 LTS
Security Type	Standard
Authentication	SSH Public Key
Network Security	Azure Network Security Group (NSG)
HTTPS Access	Enabled (TCP 443)
Wazuh Agent Communication	TCP 1514 and TCP 1515



5.4 Implementation

The Ubuntu Server virtual machine was provisioned using Microsoft Azure. Secure Shell (SSH) key-based authentication was configured to provide secure administrative access.

After the virtual machine was deployed, the necessary Network Security Group (NSG) rules were configured to allow secure communication between the Wazuh server and monitored endpoints. HTTPS access was enabled for the Wazuh Dashboard, while ports **1514** and **1515** were opened to allow Wazuh agents to communicate with the manager.

Once the network configuration was completed, the server environment was prepared for the installation of the Wazuh platform.

wazuh-ubuntuserver | Network settings ☆ ... List all my network interfaces for this VM +2 ✕

Virtual machine

Rules Collapse all

Network security group **wazuh-ubuntuserver-nsg** (attached to networkInterface: wazuh-ubuntuserver757)
Impacts 0 subnets, 1 network interfaces

+ Create port rule

Search rules Source == all Destination == all Protocol == all Action == all Port == all

P...	Name	Port	Protocol	Source	Destination
Inbound port rules (7)					
1000	default-allow-ssh	22	TCP	Any	Any
1010	Allow-HTTPS-Wazuh	443	TCP	Any	Any
1020	Allow-wazuh-1514	1514	TCP	Any	Any
1030	Allow-wazuh-1515	1515	TCP	Any	Any
65000	AllowVnetInBound	Any	Any	VirtualNetwork	VirtualNetwork
65001	AllowAzureLoadBalancerInB...	Any	Any	AzureLoadBalancer	Any

5.5 Validation

The deployment was verified by confirming:

- Successful SSH access to the Azure virtual machine.
- Network connectivity between the server and monitored endpoints.
- Accessibility of the Wazuh Dashboard through a web browser.
- Availability of the Wazuh Manager services.

5.6 Outcome

The Azure infrastructure successfully hosted the Wazuh SIEM platform and provided a stable foundation for the remaining project phases, including endpoint onboarding, file integrity monitoring and threat intelligence integration.

Microsoft Azure

Search resources, services, and docs (G+)

Copilot

Home > Compute infrastructure | Virtual machines

Compute infrastructure | Virtual machines

Virtual machines Get started

+ Create Reservations

Virtual machines

Virtual Machine Scale Set (VMSS)

Compute Fleet

Arc Machines

Disks + images

Custom images

Disks

Snapshots

Disk encryption sets

Capacity + placement

Related services

Monitoring + Operations

Showing 1 - 1 of 1. Display count: auto

wazuh-ubuntuserver Virtual machine

Help me copy this VM in any region Manage this VM with Azure CLI

Advisor (1 of 4): Migrate workload to D-series or better virtual machine

Connect Start Restart Stop Hibernate Capture Delete

Connect via Bastion

Resource group (move): SOC-WAZUH

Status: Stopped (deallocated)

Location: Central India

Subscription (move): Azure for Students

Subscription ID: 756c2ea3-9e15-4327-ae44-0afa10e8b485

Operating system: Linux

Size: Standard B2as v2 (2 vcpus, 8 GiB memory)

Primary NIC public IP: 20.219.162.174

Virtual network/subnet: vnet-centralindia/snet-centralindia-1

DNS name: Not configured

Health state: -

Time created: 1/25/2026, 12:04 PM UTC

Tags (edit): More (5)

6. Wazuh Manager Installation and Configuration

6.1 Objective

The objective of this phase was to deploy and configure the Wazuh platform on the Azure Ubuntu Server. The Wazuh Manager was installed along with the Wazuh Indexer and Wazuh Dashboard to provide centralized log collection, event analysis, storage, and visualization.

6.2 Installation Overview

The Wazuh platform was deployed on the Ubuntu Server virtual machine using the official Wazuh installation method. The deployment included all core components required to operate the SIEM platform.

The installed components were:

- Wazuh Manager
- Wazuh Indexer
- Wazuh Dashboard

These components were deployed on the same Azure virtual machine to simplify management while maintaining a centralized monitoring environment.

6.3 Component Description

Wazuh Manager

The Wazuh Manager served as the central component responsible for receiving logs from monitored endpoints, analysing security events, correlating alerts, and coordinating integrations such as VirusTotal and Active Response.

Wazuh Indexer

The Wazuh Indexer stored and indexed all collected security events, enabling efficient searching and retrieval of historical data.

Wazuh Dashboard

The Wazuh Dashboard provided a web-based interface for monitoring agents, analysing alerts, searching logs, and visualizing security events in real time.

6.4 Configuration

After installation, the following configurations were completed:

- Configured secure communication between the Wazuh components.
- Enabled HTTPS access to the Wazuh Dashboard.
- Verified that all Wazuh services started successfully.
- Confirmed that the dashboard was accessible through the Azure public IP address.

- Validated communication between the Manager, Indexer, and Dashboard.

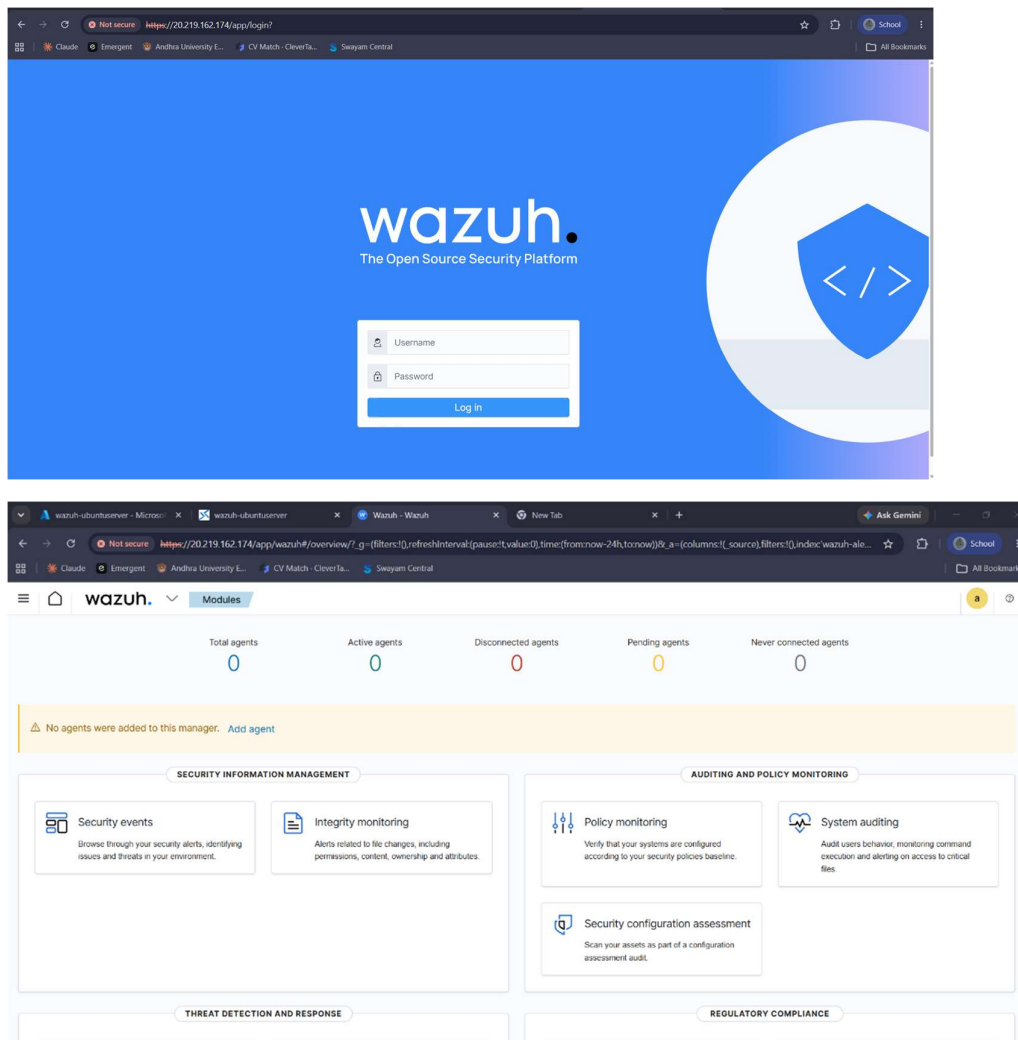
6.5 Validation

The deployment was validated by confirming the following:

- All Wazuh services were running successfully.
- The Wazuh Dashboard was accessible through a web browser.
- Administrative login to the dashboard was successful.
- No service or communication errors were observed after deployment.

6.6 Outcome

The Wazuh SIEM platform was successfully deployed and configured on the Azure Ubuntu Server. The environment was fully operational and ready to onboard monitored endpoints for security event collection and analysis.



7. Ubuntu Agent Deployment

7.1 Objective

The objective of this phase was to onboard a Linux endpoint into the Wazuh environment by installing and configuring the Wazuh Agent. This enabled the endpoint to forward security events to the Wazuh Manager for centralized monitoring.

7.2 Endpoint Overview

An Ubuntu Desktop virtual machine was used as the monitored Linux endpoint. The system communicated securely with the Wazuh Manager deployed on Microsoft Azure.

Once registered, the endpoint became visible in the Wazuh Dashboard and began forwarding security events for analysis.

7.3 Agent Installation

The Wazuh Agent was installed on the Ubuntu Desktop virtual machine using the official Wazuh agent package.

During installation:

- The Wazuh Manager IP address was configured.
- The agent was registered with the manager.
- The Wazuh Agent service was started and enabled.

After registration, the endpoint established a secure connection with the Wazuh Manager.

7.4 Configuration

The following configurations were completed:

- Configured the manager address within the agent configuration.
- Started the Wazuh Agent service.
- Enabled automatic startup during system boot.
- Verified successful registration with the Wazuh Manager.

wazuh. Agents

1 Select the package to download and install on your system:

LINUX

☐ RPM amd64
 ☐ RPM aarch64
 ☒ DEB amd64
 ☐ DEB aarch64

[For additional systems and architectures, please check our documentation.](#)

WINDOWS

☐ MSI 32/64 bits

macOS

☐ Intel
 ☐ Apple silicon

2 Server address:

2 Server address:

This is the address the agent uses to communicate with the server. Enter an IP address or a fully qualified domain name (FQDN).

Assign a server address:

Server address

3 Optional settings:

By default, the deployment uses the hostname as the agent name. Optionally, you can use a different agent name in the field below.

Assign an agent name:

Agent name

The agent name must be unique. It can't be changed once the agent has been enrolled.

Select one or more existing groups:

```

snowden@snowden-VirtualBox: ~
snowden@snowden-VirtualBox:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:33:c7:3b brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute enp0s3
        valid_lft 86276sec preferred_lft 86276sec
    inet6 fd17:625c:f037:2:a00:27ff:fe33:c73b/64 scope global dynamic noprefixroute
        valid_lft 86278sec preferred_lft 14278sec
    inet6 fe80::a00:27ff:fe33:c73b/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
snowden@snowden-VirtualBox:~$ wget https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent_4.7.5-1_amd64.deb && sudo dpkg -i ./wazuh-agent_4.7.5-1_amd64.deb
--2026-07-07 12:12:59-- https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent_4.7.5-1_amd64.deb
Resolving packages.wazuh.com (packages.wazuh.com)... 99.86.147.94, 99.86.147.123, 99.86.147.26, ...
Connecting to packages.wazuh.com (packages.wazuh.com)|99.86.147.94|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 9378818 (8.9M) [application/vnd.debian.binary-package]
Saving to: 'wazuh-agent_4.7.5-1_amd64.deb'

wazuh-agent_4.7.5-1_amd64.deb  100%[=====] 8.94M 4.06MB/s  in 2.2s

2026-07-07 12:13:02 (4.06 MB/s) - 'wazuh-agent_4.7.5-1_amd64.deb' saved [9378818/9378818]

[sudo: authenticate] Password:
Selecting previously unselected package wazuh-agent.
(Reading database... 155823 files and directories currently installed.)
Preparing to unpack ../wazuh-agent_4.7.5-1_amd64.deb...
[sudo: authenticate] Password:
Selecting previously unselected package wazuh-agent.
(Reading database... 155823 files and directories currently installed.)
Preparing to unpack ../wazuh-agent_4.7.5-1_amd64.deb...
Unpacking wazuh-agent (4.7.5-1)...
Setting up wazuh-agent (4.7.5-1)...
snowden@snowden-VirtualBox:~$ sudo systemctl daemon-reload
snowden@snowden-VirtualBox:~$ sudo systemctl enable wazuh-agent
Created symlink '/etc/systemd/system/multi-user.target.wants/wazuh-agent.service' -> '/usr/lib/systemd/system/wazuh-agent.service'.
snowden@snowden-VirtualBox:~$ sudo systemctl start wazuh-agent
snowden@snowden-VirtualBox:~$

```

7.5 Validation

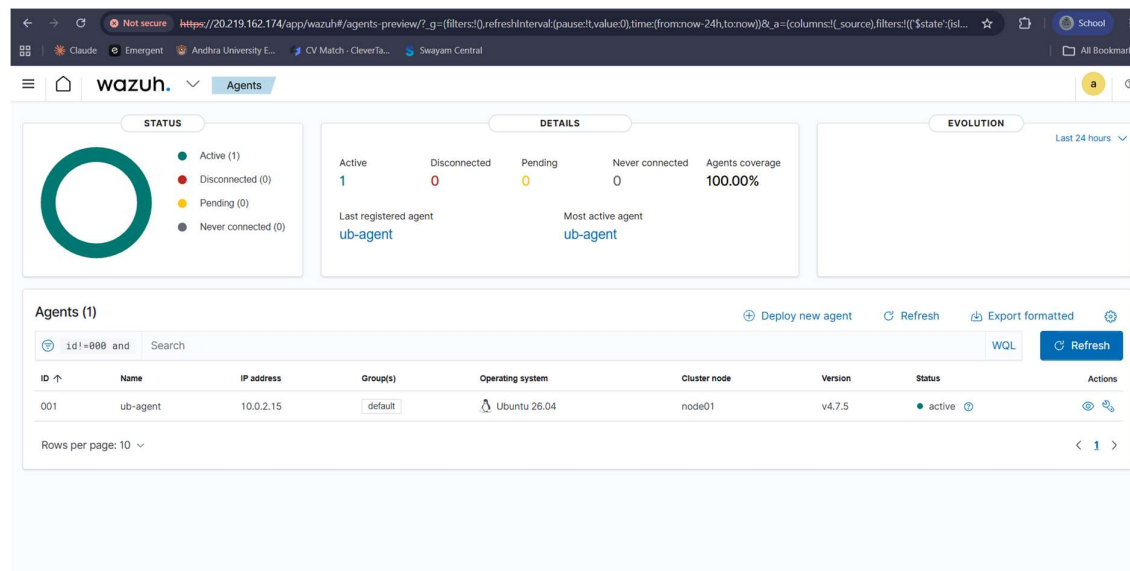
The deployment was verified by confirming:

- The Ubuntu endpoint appeared in the Wazuh Dashboard.

- The agent status showed as **Active**.
- Security events generated on the endpoint were successfully received by the Wazuh Manager.
- No communication errors were observed.

7.6 Outcome

The Ubuntu endpoint was successfully integrated into the Wazuh environment. With the agent actively forwarding security events, the endpoint was ready for advanced monitoring features, including File Integrity Monitoring (FIM) and threat intelligence integration.



8. File Integrity Monitoring (FIM)

8.1 Objective

The objective of this phase was to implement File Integrity Monitoring (FIM) to detect changes made to critical files on the monitored Linux endpoint. Wazuh was configured to monitor a designated directory and generate alerts whenever files were created, modified, renamed, or deleted.

8.2 Overview

File Integrity Monitoring is a security mechanism that continuously monitors files and directories for unauthorized changes. It enables security teams to detect suspicious file activity that may indicate malware execution, unauthorized access, or system compromise.

For this project, Wazuh FIM was configured to monitor a dedicated directory on the Ubuntu endpoint.

`/home/snowden/Desktop/monitorthis`

This directory served as the testing location for validating file monitoring events.

```

root@snowden-VirtualBox: /var/ossec/etc — sudo su
root@snowden-VirtualBox: /var/ossec/etc — sudo su
root@snowden-VirtualBox: /home/snowden/Desktop — sudo su
GNU nano 8.7.1 ossec.conf *
<interval>12h</interval>
<skip_nfs>yes</skip_nfs>
</sca>

<!-- File integrity monitoring -->
<syscheck>
<disabled>no</disabled>
<directories realtime="yes">/home/snowden/Desktop/monitorthis</directories>

<!-- Frequency that syscheck is executed default every 12 hours -->
<frequency>43200</frequency>

<scan_on_start>yes</scan_on_start>

<!-- Directories to check (perform all possible verifications) -->
<directories>/etc,/usr/bin,/usr/sbin</directories>
<directories>/bin,/sbin,/boot</directories>

<!-- Files/directories to ignore -->
<ignore>/etc/mtab</ignore>
<ignore>/etc/hosts.deny</ignore>
<ignore>/etc/mail/statistics</ignore>
<ignore>/etc/random-seed</ignore>
<ignore>/etc/random.seed</ignore>
<ignore>/etc/adjtime</ignore>
<ignore>/etc/httpd/logs</ignore>
<ignore>/etc/utmpx</ignore>

```

8.3 Configuration

The Wazuh Agent configuration was updated to enable real-time monitoring of the designated directory.

The configuration included:

- Real-time file monitoring
- Continuous integrity checking
- Monitoring of file creation, modification, renaming, and deletion events

After updating the configuration, the Wazuh Agent service was restarted to apply the changes.

8.4 Validation

To verify the implementation, several file operations were performed within the monitored directory.

The following events were tested:

Operation	Expected Result
File Creation	Wazuh generated a file creation alert
File Modification	Wazuh detected the modified file
File Rename	Wazuh recorded the rename event
File Deletion	Wazuh generated a deletion alert

Each event was successfully detected and forwarded to the Wazuh Manager, where it was displayed in the Wazuh Dashboard.

Modules ub-agent Integrity monitoring ⓘ					
					
timestamp per 30 minutes					
Time	syscheck.path	syscheck.event	rule.description	rule.level	rule.id
> Jul 7, 2026 @ 13:01:43.619	/home/snowden/Desktop/monitorthis/iamagent	modified	Integrity checksum changed.	7	550
> Jul 7, 2026 @ 13:00:57.740	/home/snowden/Desktop/monitorthis/iamagent	modified	Integrity checksum changed.	7	550
> Jul 7, 2026 @ 12:59:12.104	/home/snowden/Desktop/monitorthis/report.txt	modified	Integrity checksum changed.	7	550
> Jul 7, 2026 @ 12:58:37.049	/home/snowden/Desktop/monitorthis/report.txt	added	File added to the system.	5	554
> Jul 7, 2026 @ 12:58:37.045	/home/snowden/Desktop/monitorthis/test1.txt	deleted	File deleted.	7	553
> Jul 7, 2026 @ 12:58:11.324	/home/snowden/Desktop/monitorthis/test1.txt	added	File added to the system.	5	554
> Jul 7, 2026 @ 12:57:05.433	/home/snowden/Desktop/monitorthis/test1.txt	deleted	File deleted.	7	553
> Jul 7, 2026 @ 12:55:18.274	/home/snowden/Desktop/monitorthis/test1.txt	modified	Integrity checksum changed.	7	550
> Jul 7, 2026 @ 12:52:22.865	/home/snowden/Desktop/monitorthis/test1.txt	modified	Integrity checksum changed.	7	550
> Jul 7, 2026 @ 12:51:13.489	/home/snowden/Desktop/monitorthis/test1.txt	added	File added to the system.	5	554

8.5 Results

The implementation successfully detected all tested file operations in real time.

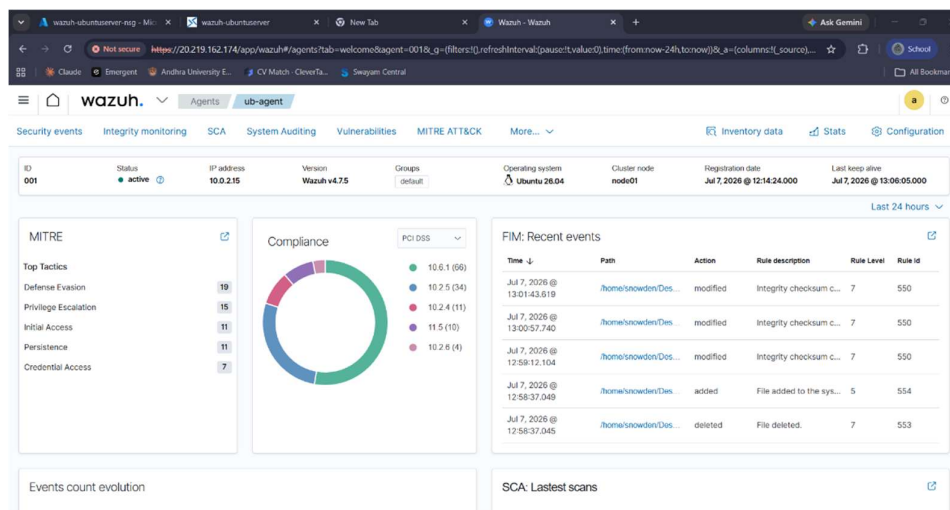
The generated alerts included information such as:

- File name
- File path
- Event type
- Timestamp
- Monitored endpoint

These alerts confirmed that the Linux endpoint was actively monitored and capable of reporting file integrity events to the Wazuh Manager.

8.6 Outcome

File Integrity Monitoring was successfully implemented and validated. This provided the foundation for integrating external threat intelligence through VirusTotal, enabling Wazuh to determine whether detected files were malicious.



9. VirusTotal Integration

9.1 Objective

The objective of this phase was to integrate Wazuh with VirusTotal to enrich file integrity events with external threat intelligence. This allowed Wazuh to automatically verify monitored file hashes against VirusTotal's malware database and generate enriched alerts for potentially malicious files.

9.2 Overview

By default, File Integrity Monitoring detects changes to monitored files but does not determine whether a file is malicious.

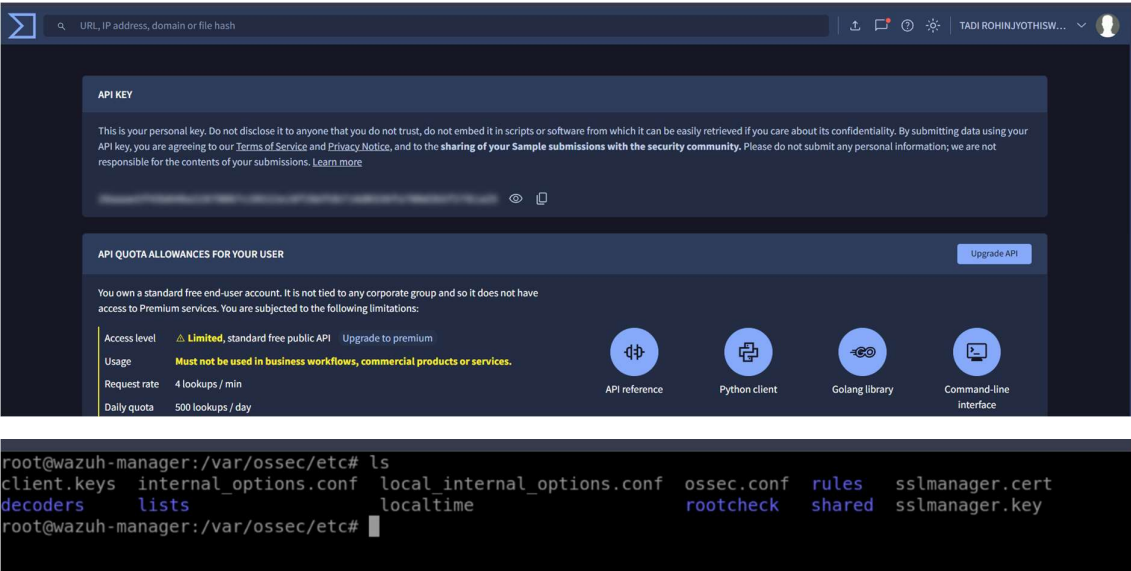
To enhance detection capabilities, Wazuh was integrated with the VirusTotal API. Whenever a new or modified file was detected, Wazuh generated its SHA-256 hash and queried VirusTotal. If the hash matched a known malicious file, the detection results were included in the security alert.

9.3 Configuration

The integration was configured by:

- Creating a VirusTotal account.
- Generating a personal VirusTotal API key.
- Configuring the VirusTotal integration within the Wazuh Manager.
- Restarting the Wazuh Manager to apply the configuration.

After the configuration was completed, Wazuh automatically queried VirusTotal for supported file integrity events.



The top part of the image shows the VirusTotal API dashboard. It includes a search bar at the top, a section for the API key with a warning about confidentiality, and a section for API quota allowances. The quota section lists limitations for a standard free end-user account: Access level is 'Limited, standard free public API', Usage is 'Must not be used in business workflows, commercial products or services.', Request rate is '4 lookups / min', and Daily quota is '500 lookups / day'. There are also links to upgrade to premium and various client libraries (API reference, Python client, Golang library, Command-line interface).

The bottom part of the image shows a terminal window with the following commands and output:

```
root@wazuh-manager:/var/ossec/etc# ls
client.keys  internal_options.conf  local_internal_options.conf  ossec.conf  rules  sslmanager.cert
decoders    lists                  localtime                  rootcheck   shared  sslmanager.key
root@wazuh-manager:/var/ossec/etc#
```

```

</localfile>

<localfile>
  <log_format>syslog</log_format>
  <location>/var/log/dpkg.log</location>
</localfile>

<localfile>
  <log_format>syslog</log_format>
  <location>/var/log/kern.log</location>
</localfile>

<integration>
  <name>virustotal</name>
  <api_key>[REDACTED]</api_key>
  <group>syscheck</group>
  <alert_format>json</alert_format>
</integration>

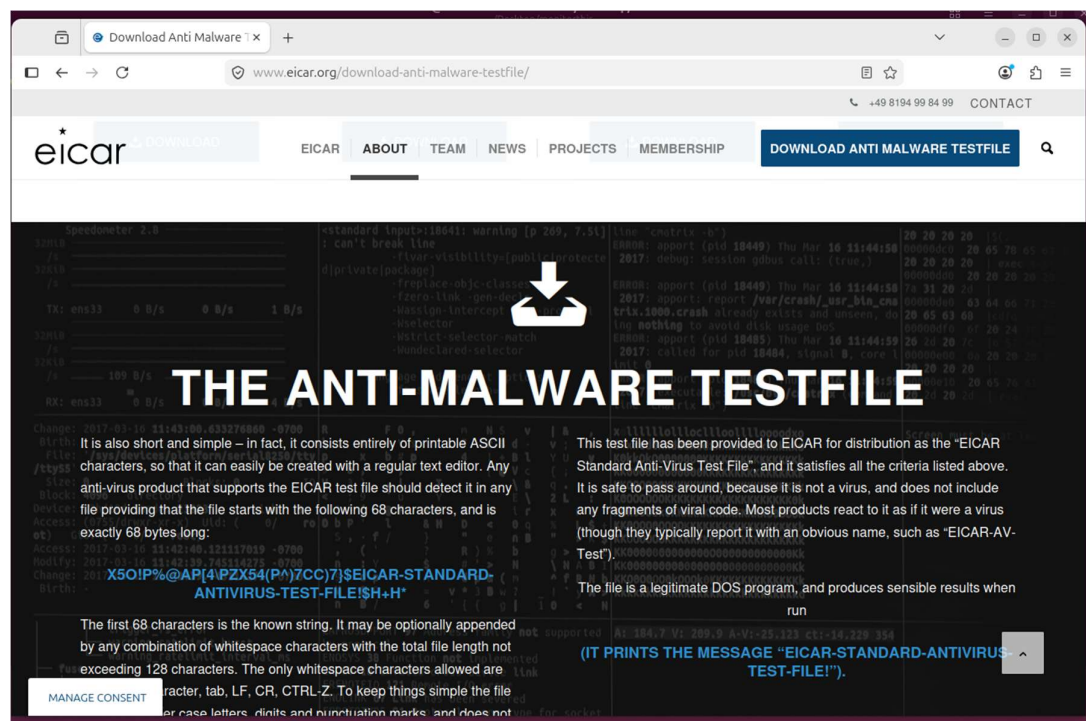
```

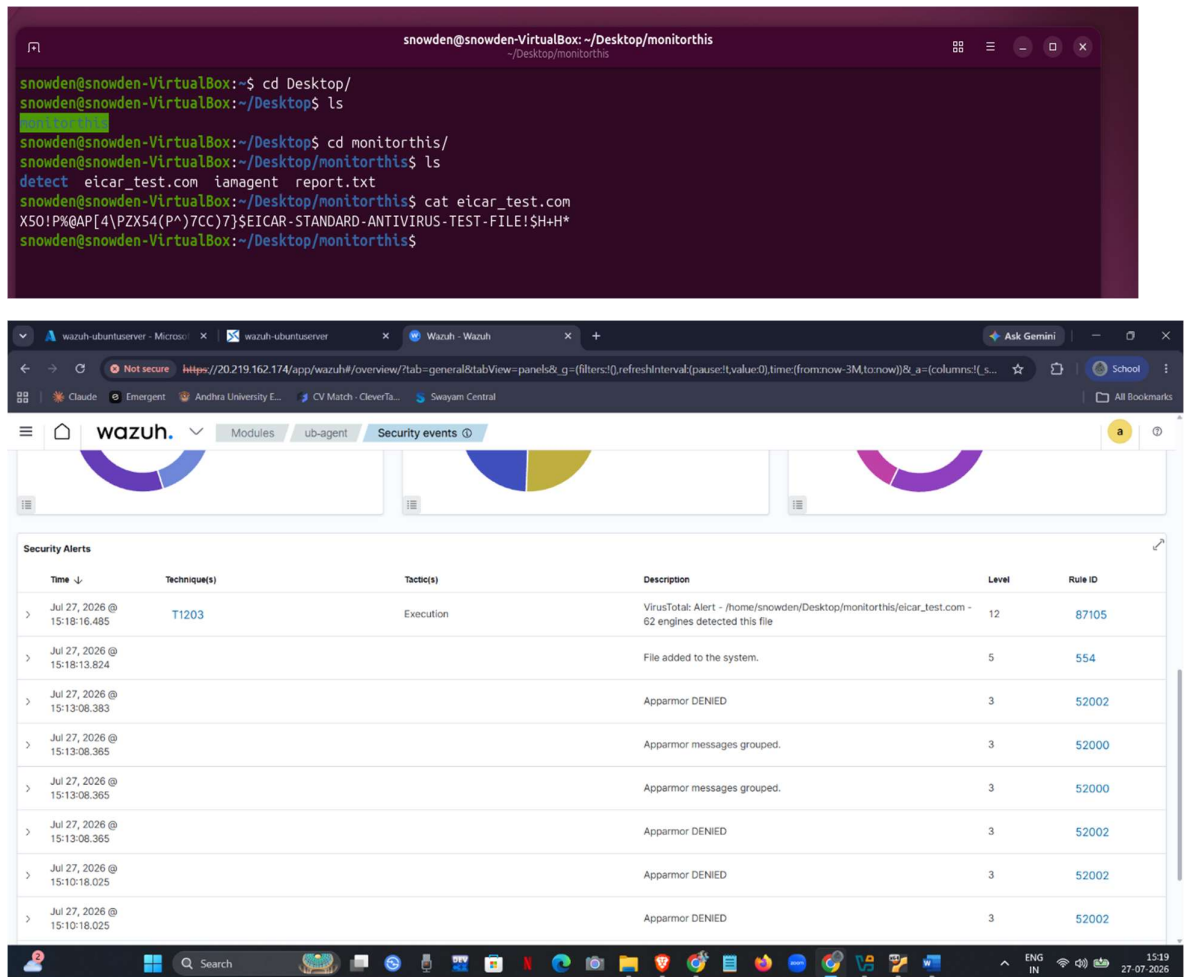
9.4 Validation

To verify the integration, the **EICAR Anti-Malware Test File** was placed inside the monitored directory.

The following workflow was successfully validated:

1. Wazuh FIM detected the newly created file.
2. The file hash was generated.
3. Wazuh queried the VirusTotal API.
4. VirusTotal identified the file as malicious.
5. Wazuh generated an enriched security alert displaying the VirusTotal detection results.





9.5 Results

The VirusTotal integration functioned successfully and enriched Wazuh alerts with external threat intelligence.

The generated alert included:

- File name
- File path
- Detection count
- VirusTotal analysis
- Alert severity
- Timestamp

The successful detection of the EICAR test file confirmed that the integration was operating correctly and that malicious files could be identified using external threat intelligence.

9.6 Outcome

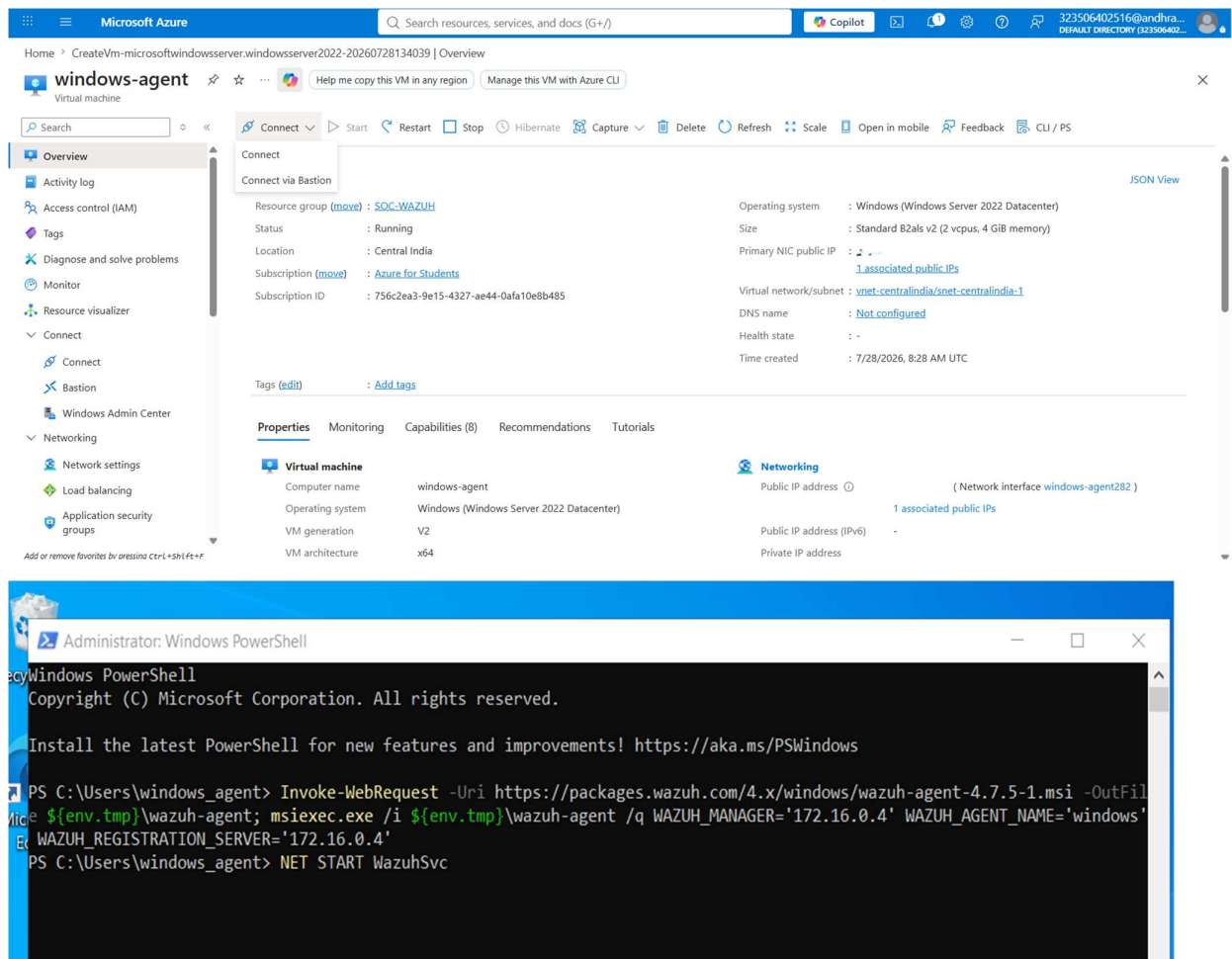
The integration significantly improved the SOC's detection capabilities by combining local file integrity monitoring with cloud-based malware intelligence. This enabled the environment to distinguish between ordinary file changes and files that were recognised as malicious by VirusTotal.

10. Windows Endpoint Monitoring

10.1 Objective

The objective of this phase was to onboard a Windows endpoint into the Wazuh SIEM environment and configure advanced endpoint monitoring capabilities. A Windows virtual machine was connected to the Wazuh Manager using the Wazuh Agent. To improve visibility into Windows activities, Microsoft Sysmon was deployed, and File Integrity Monitoring (FIM) was configured to detect file system changes within a designated directory.

This phase enabled centralized monitoring of Windows security events alongside the existing Linux endpoint, providing comprehensive endpoint visibility within the SOC environment.



10.2 Windows Agent Deployment

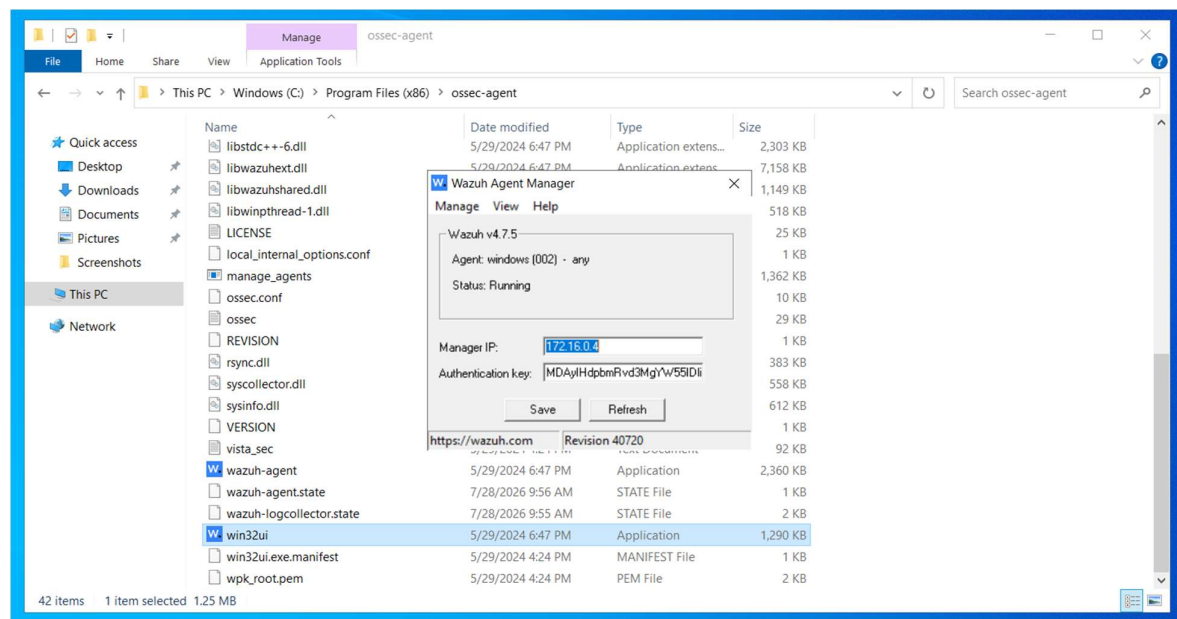
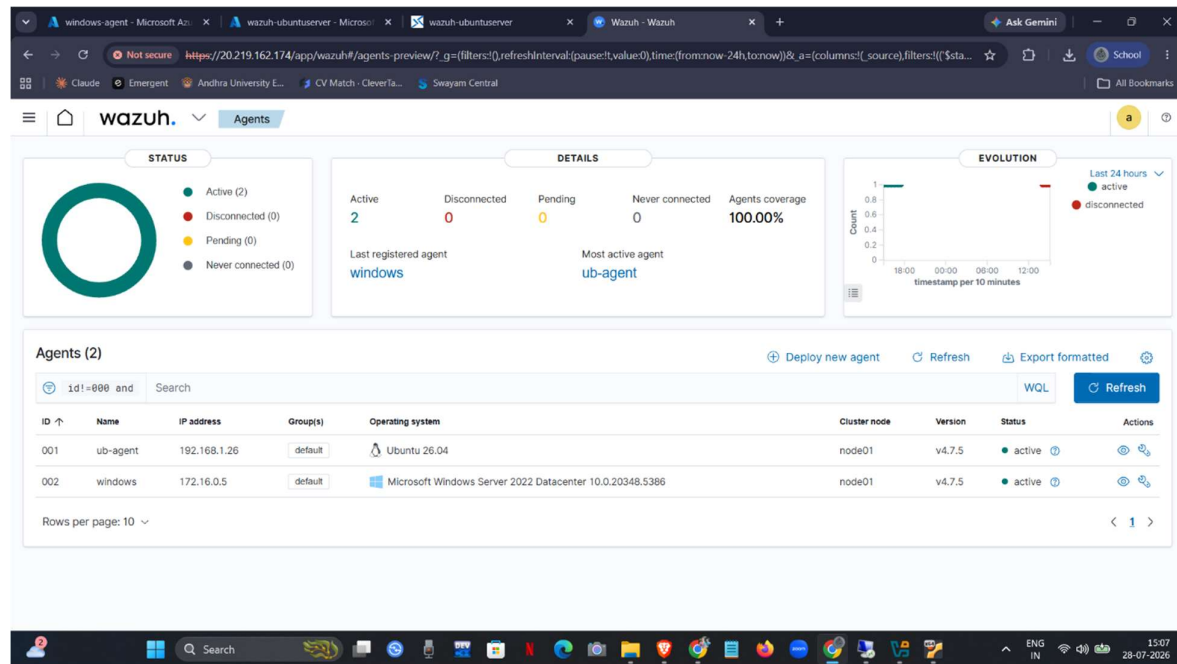
A Microsoft Windows virtual machine hosted in Microsoft Azure was deployed as the monitored Windows endpoint. The Wazuh Agent was installed using the official Wazuh deployment package and configured to communicate securely with the Wazuh Manager hosted on the Ubuntu Server.

During the deployment, the following tasks were completed:

- Installed the Wazuh Agent on the Windows endpoint.

- Configured the Wazuh Manager address within the agent configuration.
- Registered the Windows endpoint with the Wazuh Manager.
- Started the Wazuh Agent service.
- Verified successful communication between the Windows endpoint and the Wazuh Dashboard.

Following registration, the Windows endpoint appeared as an active agent in the Wazuh Dashboard and began forwarding Windows event logs to the manager for centralized analysis.

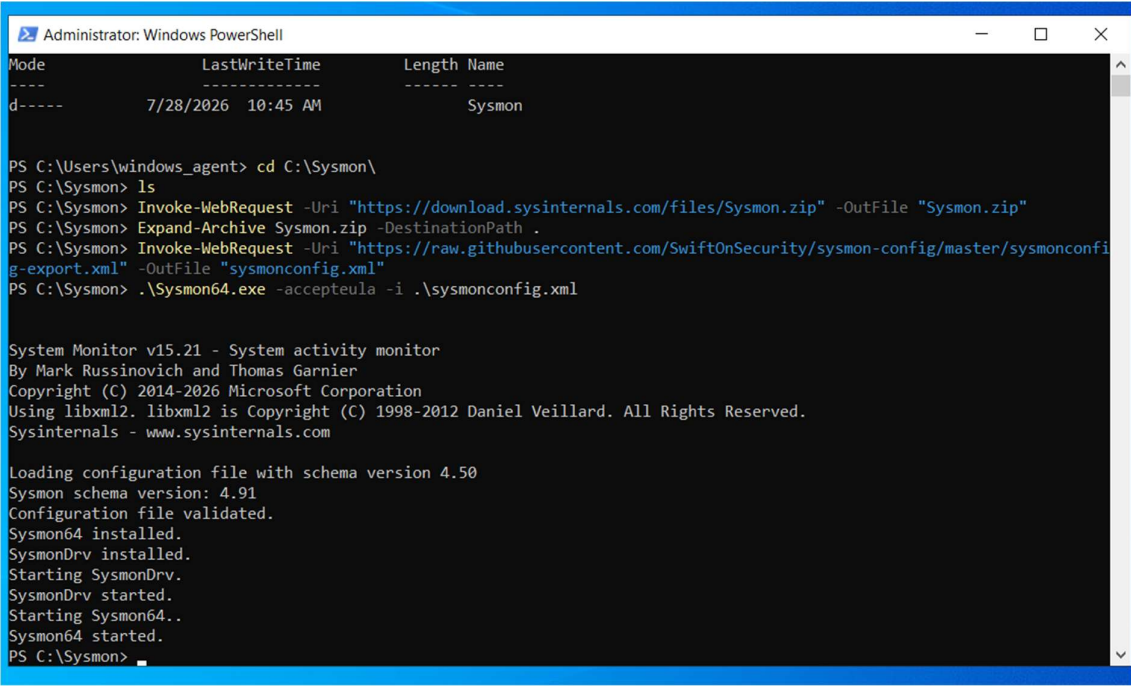


10.3 Sysmon Integration

To enhance Windows endpoint monitoring, Microsoft Sysmon (System Monitor) was installed on the Windows virtual machine. Unlike standard Windows Event Logs, Sysmon provides detailed security telemetry, including process creation, network connections, driver loading, registry modifications, and file creation events.

The SwiftOnSecurity Sysmon configuration was deployed to generate meaningful security events while minimizing unnecessary log generation. The Wazuh Agent was configured to monitor the **Microsoft-Windows-Sysmon/Operational** event channel, allowing Sysmon events to be forwarded to the Wazuh Manager for centralized processing and analysis.

The integration significantly improved endpoint visibility by providing detailed process execution information that supports security investigations and threat detection.



```

Administrator: Windows PowerShell

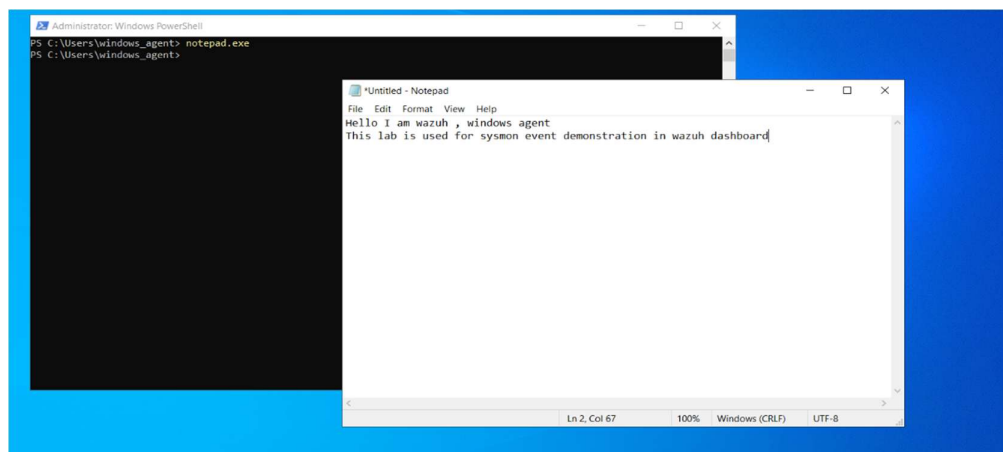
Mode                LastWriteTime         Length Name
-----
d-----          7/28/2026  10:45 AM             Sysmon

PS C:\Users\windows_agent> cd C:\Sysmon\
PS C:\Sysmon> ls
PS C:\Sysmon> Invoke-WebRequest -Uri "https://download.sysinternals.com/files/Sysmon.zip" -OutFile "Sysmon.zip"
PS C:\Sysmon> Expand-Archive Sysmon.zip -DestinationPath .
PS C:\Sysmon> Invoke-WebRequest -Uri "https://raw.githubusercontent.com/SwiftOnSecurity/sysmon-config/master/sysmonconfig-export.xml" -OutFile "sysmonconfig.xml"
PS C:\Sysmon> .\Sysmon64.exe -accepteula -i .\sysmonconfig.xml

System Monitor v15.21 - System activity monitor
By Mark Russinovich and Thomas Garnier
Copyright (C) 2014-2026 Microsoft Corporation
Using libxml2. libxml2 is Copyright (C) 1998-2012 Daniel Veillard. All Rights Reserved.
Sysinternals - www.sysinternals.com

Loading configuration file with schema version 4.50
Sysmon schema version: 4.91
Configuration file validated.
Sysmon64 installed.
SysmonDrv installed.
Starting SysmonDrv.
SysmonDrv started.
Starting Sysmon64..
Sysmon64 started.
PS C:\Sysmon>
  
```

Following the installation, Sysmon-generated events were successfully received and processed by Wazuh, confirming that enhanced Windows telemetry was functioning correctly.



Sysmon events successfully collected and displayed within the Wazuh Dashboard.

wazuh.

ModuleswindowsSecurity events ⓘ

Jul 28, 2026 @ 18:33:36.267

T1105

Command and Control

Executable file dropped in folder commonly used by malware

TableJSONRule

@timestamp	2026-07-28T13:03:36.267Z
_id	-EJSqJ8B2PCMHsdY7a9e
agent.id	002
agent.ip	172.16.0.5
agent.name	windows
data.win.eventdata.creationUtcTime	2026-07-28 13:03:35.531
data.win.eventdata.image	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe
data.win.eventdata.processGuid	{f7e41ce4-a8a7-6a68-b803-00000000400}
data.win.eventdata.processId	3400
data.win.eventdata.targetFilename	C:\Users\windows_agent\AppData\Local\Temp\2_PSScriptPolicyTest_rk0nlv3i.2pb.ps1
data.win.eventdata.user	windows-agent\windows_agent
data.win.eventdata.utcTime	2026-07-28 13:03:35.531
data.win.system.channel	Microsoft-Windows-Sysmon/Operational
data.win.system.computer	windows-agent

data.win.system.message

"File created:
RuleName: -
UtcTime: 2026-07-28 13:03:35.531
ProcessGuid: {f7e41ce4-a8a7-6a68-b803-00000000400}
ProcessId: 3400
Image: C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe
TargetFilename: C:\Users\windows_agent\AppData\Local\Temp\2_PSScriptPolicyTest_rk0nlv3i.2pb.ps1
CreationUtcTime: 2026-07-28 13:03:35.531
User: windows-agent\windows_agent"

data.win.system.opcode	0
data.win.system.processID	3212
data.win.system.providerGuid	{5770385f-c22a-43e0-bf4c-06f5698ffbd9}
data.win.system.providerName	Microsoft-Windows-Sysmon
data.win.system.severityValue	INFORMATION
data.win.system.systemTime	2026-07-28T13:03:35.5437804Z
data.win.system.task	11
data.win.system.threadID	5616
data.win.system.version	2
decoder.name	windows_eventchannel

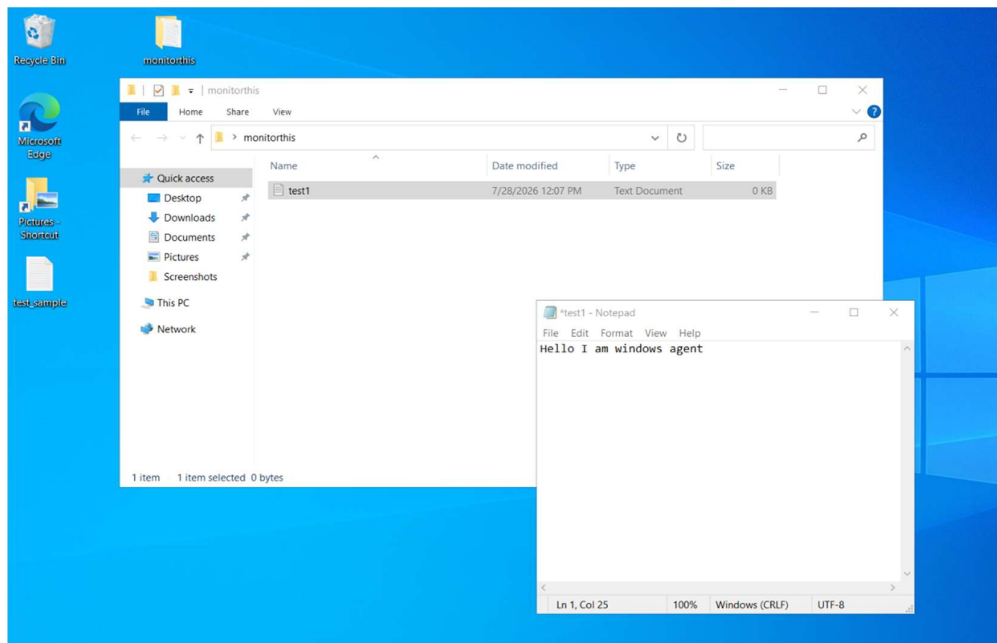
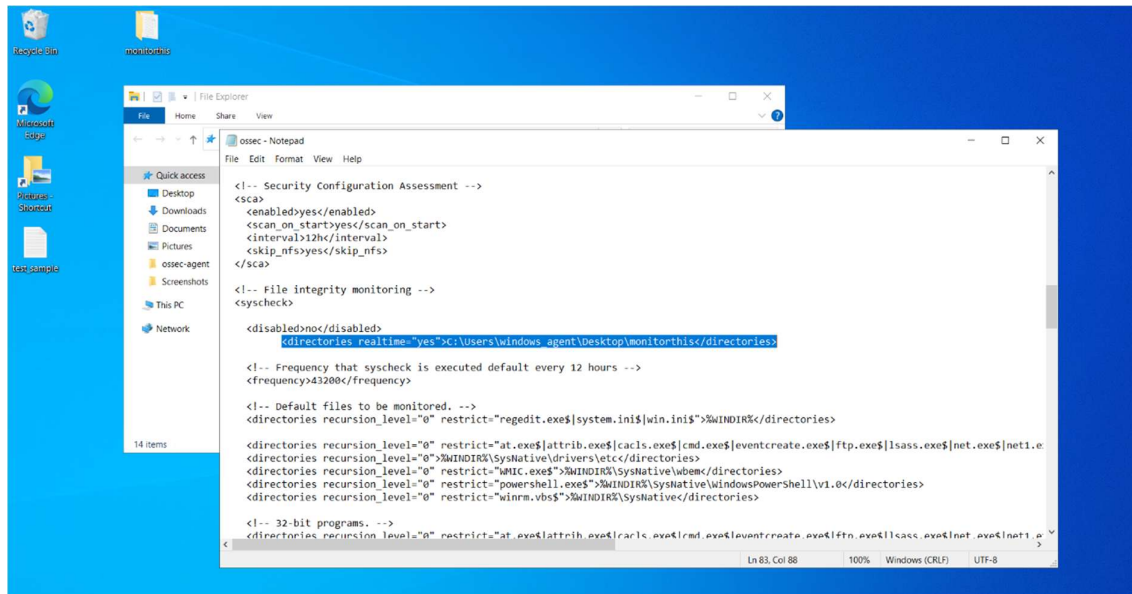
10.4 Windows File Integrity Monitoring

After establishing communication between the Windows endpoint and the Wazuh Manager, File Integrity Monitoring (FIM) was configured to monitor changes occurring within a dedicated directory.

The monitored directory was configured as:

C:\Users\windows_agent\Desktop\monitorthis

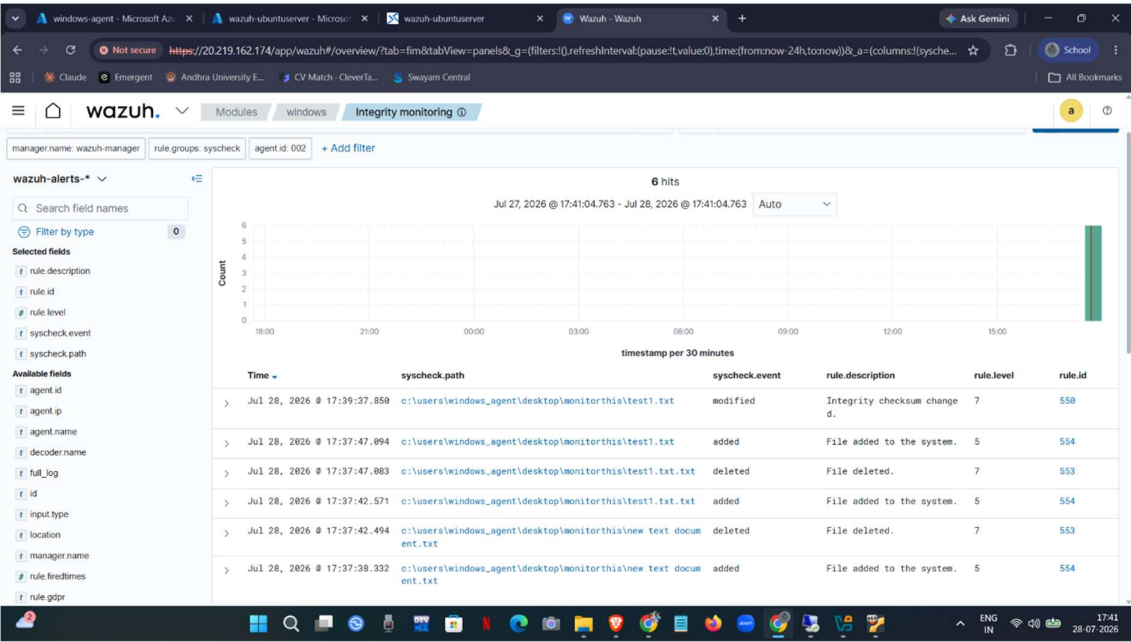
The directory was added to the Windows agent configuration with real-time monitoring enabled. After updating the configuration, the Wazuh Agent service was restarted to apply the new monitoring policy.



The implementation continuously monitored the directory for:

- File creation
- File modification
- File deletion

Whenever any monitored activity occurred, the Wazuh Agent transmitted the event to the Wazuh Manager, where it was processed and displayed within the Wazuh Dashboard.



10.5 Validation

To validate the Windows endpoint monitoring implementation, several file operations were performed within the monitored directory.

A test file was created, modified, and subsequently deleted. Each operation generated the corresponding File Integrity Monitoring alert within the Wazuh Dashboard.

The generated alerts included:

File Operation	Wazuh Detection
File Creation	File Added
File Modification	Integrity Checksum Changed
File Deletion	File Deleted

These alerts confirmed that the Windows endpoint was successfully forwarding real-time file integrity events to the centralized SIEM platform.

10.6 Outcome

The Windows endpoint was successfully integrated into the cloud-based SOC environment using the Wazuh Agent. Secure communication between the endpoint and the Wazuh Manager was established, enabling centralized collection and analysis of Windows security events.

The deployment of Microsoft Sysmon enhanced endpoint visibility by providing detailed system telemetry, while Windows File Integrity Monitoring enabled real-time detection of unauthorized file system changes. Together, these components strengthened the monitoring capabilities of the SIEM platform by extending comprehensive visibility across both Linux and Windows endpoints.

The successful implementation of Windows endpoint monitoring completed the endpoint integration phase of the project and prepared the environment for end-to-end security validation and analysis.

11. Challenges Encountered

One of the initial challenges was configuring the Azure Network Security Group (NSG) to allow secure access to the Wazuh Dashboard. Appropriate inbound rules were added to permit HTTPS access and agent communication, enabling remote management of the SIEM platform.

Another challenge involved establishing reliable communication between the monitored endpoints and the Wazuh Manager. This was resolved by verifying the agent configurations, ensuring the correct manager address was specified, and confirming that all required Wazuh services were running successfully.

Integrating File Integrity Monitoring on both the Linux and Windows endpoints required updating the Wazuh Agent configuration to monitor specific directories in real time. After restarting the agent services, file creation, modification, and deletion events were successfully detected and displayed in the Wazuh Dashboard.

The integration of Microsoft Sysmon required additional configuration to forward Sysmon Operational logs to the Wazuh Manager. Once the appropriate event channel was configured, detailed Windows process and system activity became available within the SIEM dashboard, improving endpoint visibility.

Despite these challenges, all issues were successfully resolved through systematic troubleshooting and configuration validation. The completed implementation resulted in a stable cloud-based SIEM environment capable of centralized security monitoring across both Linux and Windows endpoints.

12. Results and Discussion

The implementation successfully achieved the objectives defined at the beginning of the project by deploying a cloud-based Security Operations Center (SOC) using the Wazuh SIEM platform on Microsoft Azure. The environment provided centralized security monitoring for both Linux and Windows endpoints, enabling the collection, analysis, and visualization of security events through a single management interface.

The deployment of Wazuh Agents on both endpoints established secure communication with the Wazuh Manager, allowing endpoint activities to be continuously monitored in real time. File Integrity Monitoring (FIM) was successfully implemented on both operating systems, where file creation, modification, and deletion events were accurately detected and reported to the Wazuh Dashboard. This demonstrated the platform's ability to monitor critical file system changes and improve endpoint visibility.

The integration of Microsoft Sysmon further enhanced Windows endpoint monitoring by generating detailed system telemetry. Process creation and other system activities were successfully collected and forwarded to the Wazuh Manager, providing additional context for security analysis and strengthening the overall monitoring capabilities of the SOC environment.

The VirusTotal integration extended the detection capabilities of the SIEM by enriching file integrity events with external threat intelligence. During validation, the EICAR Anti-Malware Test File was correctly identified as malicious, and Wazuh generated enriched alerts containing VirusTotal detection results. This demonstrated the effectiveness of combining local monitoring with cloud-based threat intelligence to improve malware detection.

The current implementation prioritizes core SIEM capabilities. Future enhancements focus on expanding the environment towards a complete SOC ecosystem through automation, incident management, and advanced detection.

Overall, the implemented solution provided a stable and functional SIEM environment capable of centralized log collection, endpoint monitoring, file integrity verification, enhanced Windows telemetry, and threat intelligence integration. The project successfully demonstrated how cloud infrastructure and open-source security technologies can be integrated to build a practical SOC capable of supporting real-time security monitoring and threat detection.

13. Conclusion and Future Enhancements

13.1 Conclusion

This project demonstrates the practical implementation of a cloud-based Security Operations Center (SOC) using the Wazuh Security Information and Event Management (SIEM) platform hosted on Microsoft Azure. Rather than focusing solely on tool deployment, the implementation establishes a centralized monitoring environment capable of collecting, correlating, and analysing security events generated from both Linux and Windows endpoints.

The completed SOC environment integrates multiple security capabilities into a unified monitoring platform. File Integrity Monitoring provides continuous visibility into critical file system changes, while Microsoft Sysmon enhances Windows endpoint telemetry by capturing detailed process and system activities. The integration of VirusTotal further strengthens the detection workflow by enriching file integrity events with external threat intelligence, allowing potentially malicious files to be identified and investigated more effectively.

Throughout the implementation, emphasis was placed on designing an architecture that reflects practical security operations rather than isolated feature demonstrations. By combining cloud infrastructure, endpoint monitoring, threat intelligence, and centralized event analysis, the project illustrates how open-source security technologies can be integrated to improve security visibility across heterogeneous environments.

Beyond the technical implementation, the project provided practical experience in deploying, configuring, troubleshooting, and validating a production-oriented SIEM environment. The knowledge gained extends beyond individual tools to the broader understanding of how security monitoring components interact to support continuous detection and analysis within a modern Security Operations Center.

The resulting implementation provides a scalable foundation that can be extended with additional endpoints, detection rules, and security integrations, making it a suitable reference architecture for learning, research, and future enterprise-scale SOC deployments.

13.2 Future Enhancements

The implemented SOC provides a strong foundation for centralized security monitoring; however, several capabilities can be incorporated in future work to further enhance its effectiveness.

One significant enhancement would be the implementation of **Active Response**, enabling the Wazuh Manager to automatically perform predefined actions, such as isolating compromised endpoints or removing malicious files when specific detection rules are triggered. This would extend the SOC beyond monitoring and detection to include automated incident response.

Another improvement would be the integration of **TheHive** as a case management platform. This would allow security alerts generated by Wazuh to be converted into investigation cases,

improving incident tracking, analyst collaboration, and documentation throughout the incident response lifecycle.

The implementation can also be enhanced by incorporating **Shuffle** for Security Orchestration, Automation, and Response (SOAR). Integrating Shuffle would enable automated workflows between Wazuh and external security tools, reducing manual effort and improving the efficiency of repetitive security operations.

Finally, the SOC can be extended with **custom detection rules** tailored to organisational requirements. Developing organisation-specific correlation rules would improve detection accuracy by identifying suspicious activities that may not be covered by the default Wazuh rule set.

These enhancements would transform the current implementation from a centralized monitoring platform into a more comprehensive SOC capable of automated response, structured incident management, security compliance assessment, and advanced threat detection.

14. References

1. *Wazuh Documentation*. <https://documentation.wazuh.com/>
2. *Wazuh proof of Concept*. <https://documentation.wazuh.com/current/proof-of-concept-guide/index.html>
3. *VirusTotal API Documentation*. <https://docs.virustotal.com/>
4. Microsoft Sysinternals. *Sysmon*.
<https://learn.microsoft.com/sysinternals/downloads/sysmon>
5. *Sysmon Configuration*. <https://github.com/SwiftOnSecurity/sysmon-config>